

ორმაგი აგენტის დილემა

- დროის ლიმიტი: 12 წუთი.
- მეხსიერება: 5 გბ
- გარემო: ერთი GPU (≈ 16 გბ VRAM), ინტერნეტის გარეშე
- ამოხსნის ზომა: `solution.ipynb` ≤ 1 მბ
- საბაზისო ქულა: 0

ასტანაში მდებარე AI-ის ეროვნულ ცენტრში ორი კომპიუტერული მოდელი — Model R (ResNet-18) და Model V (ViT-Tiny) — ფოტოებს აანალიზებს. ამჟამად ორივე მოდელი სრულყოფილად მუშაობს, აღწევს 100%-იან სიზუსტეს და ეთანხმება ერთმანეთს თითოეულ ფოტოზე. იმის შესამოწმებლად, თუ რამდენად განსხვავდება მათი ჭკვიანი „ტვინები“ სინამდვილეში, მთავარი მეცნიერი გთავაზობთ გამოწვევას: შეიტანეთ მცირე, თითქმის შეუმჩნეველი პიქსელური ცვლილებები თითოეულ ფოტოში ისე, რომ Model R და Model V სრულად არ დაეთანხმონ ერთმანეთს.



1. დავალება

ორი წინასწარ დატრენინგებული (pretrained) გამოსახულების კლასიფიკატორი უყურებს ერთსა და იმავე გამოსახულებას. ამ დავალებაში მოწოდებულ გამოსახულებებზე ორივე კლასიფიკატორი 100%-იანი სიზუსტით მუშაობს.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `timm-ის vit_tiny_patch16_224` (ტრანსფორმერი, ViT-Tiny).

თქვენი დავალებაა შექმნათ მცირე ცვლილება („პერტურბაცია“ / *perturbation*) თითოეული გამოსახულებისთვის ისე, რომ ორ მოდელს შორის უთანხმოება წარმოიშვას. თითოეული გამოსახულებისთვის უნდა შექმნათ **ორი განსხვავებული** პერტურბაცია:

- **A ტიპი:** მისი დამატების შემდეგ Model R კვლავ სწორად ახდენს გამოსახულების კლასიფიკაციას, ხოლო Model V არასწორად ახდენს მას.
- **B ტიპი:** მისი დამატების შემდეგ Model V კვლავ სწორად ახდენს გამოსახულების კლასიფიკაციას, ხოლო Model R არასწორად ახდენს მას.

თითოეული პერტურბაცია საკმარისად *მცირე* უნდა იყოს იმისთვის, რომ მისი შემჩნევა რთული იყოს. უფრო მცირე პერტურბაციები იღებს უფრო მაღალ ქულას (იხილეთ მე-5 სექცია). პერტურბაცია გამოიყენება ორიგინალ გამოსახულებაზე უშუალოდ პიქსელების დონეზე.

2. საჯარო მონაცემები

დავალებას მოჰყვება გამოსახულებათა სიმრავლე, რომელიც ორ დაყოფად (*split*) არის ორგანიზებული — `train` (100 გამოსახულება) და `test_public` (100 გამოსახულება) — თითოეული განსხვავებული გარჩევადობის (*resolution*) გამოსახულებებით. ყველა გამოსახულება არის ImageNet-1K-ის 1000 კლასიდან და როგორც Model R, ისე Model V აღწევს 100%-იან სიზუსტეს ორივე დაყოფაზე.

მოწოდებულია შემდეგი ფაილები:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

შეფასებისას, თქვენი `dataset/test_public/` საქაღალდე ოფიციალური შეფასებისთვის გამჭვირვალედ ჩანაცვლდება გამოსახულებათა ორი ფარული სიმრავლით (`test_leaderboard_a` და `test_leaderboard_b`). თითოეული მათგანი შეიცავს **100 გამოსახულებას** PNG ფორმატში და ლეიბლების ფაილს.

შენიშვნა: ამ დავალებისთვის, ტესტურ მონაცემთა სიმრავლეებში არსებული ლეიბლები ხელმისაწვდომია.

3. გამოსასვლელი ფორმატი

თითოეული გამოსახულებისთვის უნდა შექმნათ ორი ფაილი:

```
{index}_a.pt    # Type A perturbation
{index}_b.pt    # Type B perturbation
```

- `{index}` (0, 1, 2, ...), ემთხვევა გამოსახულების სახელს მონაცემთა სიმრავლეებში.
- თითოეული ფაილი არის `torch.save`-ით შენახული ერთი ტენზორი. მისი ზომა (shape) უნდა იყოს $3 \times H \times W$, სადაც H და W ემთხვევა ამ გამოსახულების **ორიგინალ** გარჩევადობას (და არა 224×224 -ს).
- კოდმა უნდა შექმნას მხოლოდ ერთი ZIP ფაილი, `submission.zip`. განათავსეთ ყველა `.pt` ფაილი ZIP არქივის ძირითად დონეზე, ყოველგვარი შემცველი საქაღალდისა თუ ქვესაქაღალდის გარეშე.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

ნოთბუქი გაფრთხილებით, თუ გამოსასვლელი ფორმატის რაიმე პრობლემა იქნება.

4. შეზღუდვები

- **მოდელები:** უნდა გამოიყენოთ `torchvision.models.resnet18(pretrained=True)` და `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. სხვა წინასწარ დატრენინგებული მოდელების გამოყენება დაშვებული არ არის.
- **გარდაქმნის ფაიფლაინი (სავალდებულოა შეფასებისას):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` დეტალებისთვის.
- **პერტურბაციის გარჩევადობა:** უნდა ემთხვეოდეს ნედლი გამოსახულების **ორიგინალ** გარჩევადობას (და არა 224×224 -ს). ტენზორი ემატება ნედლ გამოსახულებას გარდაქმნის ფაიფლაინამდე (*before*).
- **გამოსასვლელი ფორმატი:** მხოლოდ `.pt` ფაილები — არა PNG/JPG. ტენზორები ემატება ნედლ გამოსახულებას და პიქსელების მნიშვნელობები იჭრება `[0, 1]` ინტერვალში წინასწარ დამუშავებამდე.
- **ფაილების სახელდება:** ერთ დონეზე ჩამოთვლილი, მკაცრი `{index}_a.pt / {index}_b.pt` ფორმატი. ZIP-ის შიგნით ქვესაქაღალდეების გარეშე.
- **ბიბლიოთეკები:** `torch`, `torchvision`, `timm`.

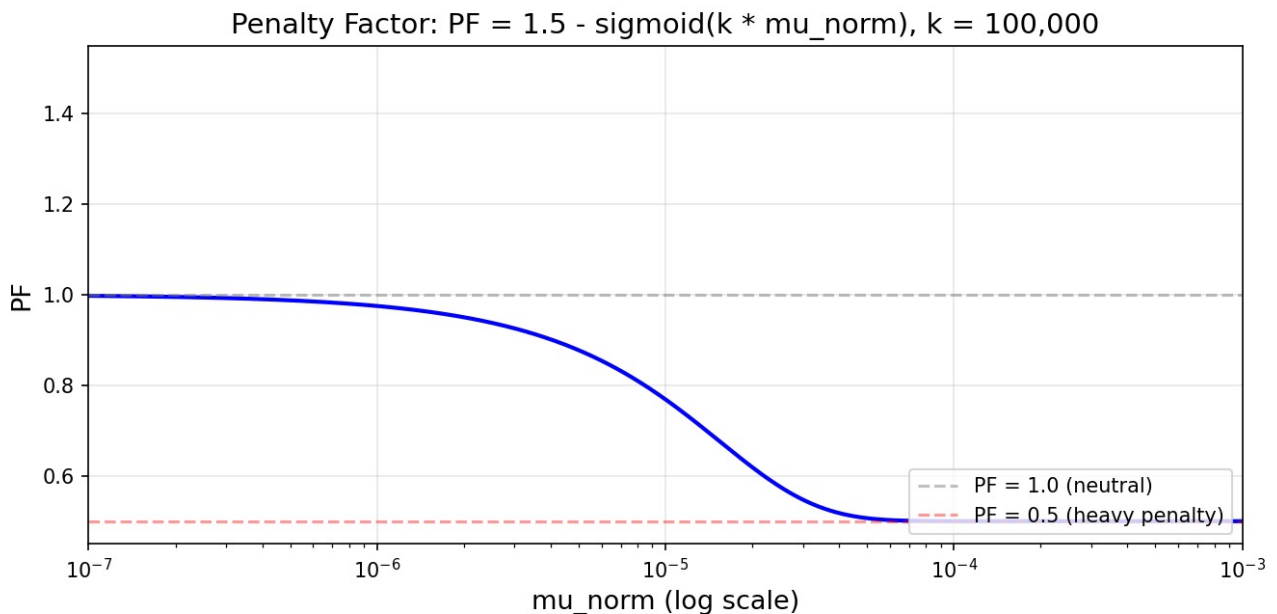
5. შეფასება

საბოლოო ქულა გამოითვლება შემდეგნაირად. იყოს M გამოსახულებათა რაოდენობა დაყოფაში, $Score_A$ წარმატებული A ტიპის პერტურბაციების რაოდენობა, ხოლო $Score_B$ წარმატებული B ტიპის

პერტურბაციების რაოდენობა:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF არის ფუნქცია, რომელიც შექმნილია მაღალი ნორმის მქონე პერტურბაციების დასასჯელად და არის ძალიან მგრძნობიარე წარმადობის მაქსიმუმთან ახლოს. იგი შეზღუდულია 0.5-დან 1-მდე დიაპაზონში. სრული იმპლემენტაცია შეგიძლიათ იხილოთ `solution.ipynb`-ის მე-8 სექციაში.



სურათი: ჯარიმის ფუნქციის მრუდი.

6. გაგზავნილის შემოწმება

ნოტბუქში არის შემოწმებები, რომლებიც გაფრთხილებთ, თუ არსებობს ფორმატირების პრობლემები, `solution.ipynb` ნოტბუქის მე-7 სექციაში.

7. ლოკალური ტესტირება

`solution.ipynb` შეიცავს სრულ, მომუშავე მაგალითს. იგი ტვირთავს საჯარო მონაცემებს, ორივე მოდელსა და ოფიციალურ შეფასების კოდს და წერს გასაგზავნ ZIP ფაილს. წაიკითხეთ იგი მუშაობის დაწყებამდე.

8. როგორ გავაგზავნოთ

- შეინახეთ თქვენი ცვლილებები `solution.ipynb` ფაილში.
- გახსენით Git ტაბი JupyterLab-ის მარცხენა გვერდითა პანელზე.
- **დაასთეიჯეთ (Stage)** `solution.ipynb` (მის გვერდით არსებული + იკონი).
- შეიყვანეთ კომიტის შეტყობინება და დააწკაპუნეთ **Commit**.
- დააწკაპუნეთ ღრუბლისა და ზემოთ მიმართული ისრის იკონს დასაბუშად.

- დაბრუნდით კონკურსის ამ გვერდზე და დააწკაპუნეთ **Submit**.

გააგზავნეთ ზუსტად ერთი ფაილი, სახელწოდებით `solution.ipynb`.